AMDARI

Implementing Scalable Identity & Access Management (IAM)

FOR: CloudScale Logistics

BY: CHINAZA NNAMANI

App/Cloud Security Consultant

Date: MAY 27, 2026

AMDARI

EXECUTIVE SUMMARY

Project Overview

This document outlines the strategic design and deployment of a Scalable Identity and Access Management (IAM) framework for CloudScale Logistics. As the organization continues to expand its cloud infrastructure and workforce, legacy access models require modernization to ensure secure, efficient, and auditable identity governance. This project successfully established a robust, future-proof IAM architecture designed to protect critical logistics data while enabling seamless, secure access for authorized personnel.

Key Implementations

- **Role-Based Access Control (RBAC):** Restructured user permissions across the organization by enforcing the Principle of Least Privilege (PoLP). Access is now strictly mapped to specific job functions, preventing lateral movement and unauthorized data exposure.
- **Centralized Authentication (SSO & MFA):** Deployed enterprise-wide Single Sign-On (SSO) integrated with Multi-Factor Authentication (MFA). This significantly reduces password fatigue for the workforce while neutralizing credential-based attacks.
- **Automated Identity Lifecycle Management:** Streamlined the onboarding and offboarding workflows. Automated provisioning and deprovisioning scripts ensure new hires get immediate access, while departing employees are instantly revoked, eliminating the risk of orphaned accounts.
- **Cloud-Native Scalability:** Designed the IAM architecture to integrate directly with CloudScale Logistics' existing cloud environments, ensuring the access model can scale dynamically alongside sudden workforce expansions or seasonal surges.

Business Impact

The deployment of this scalable IAM solution drastically reduces CloudScale Logistics' internal and external attack surface, ensuring compliance with modern data protection standards. By automating identity workflows and centralizing access controls, the organization now benefits from reduced IT administrative overhead, enhanced security posture, and a frictionless authentication experience for its entire workforce.

Phase 1 (GUI): Utilize the Azure Portal to create Users, the Senior Admins group, and assign Joseph Myers.

Environment: Microsoft Azure Portal

Objectives:

In Microsoft Entra ID, create a user account for Joseph Myers, Clara Garcia, and Bordock Williams.

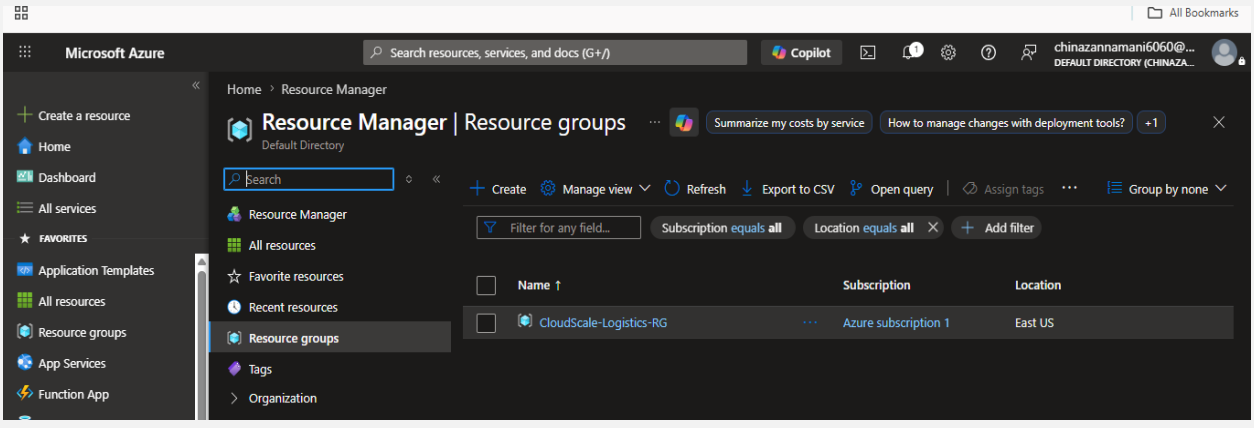
Audit the environment for "Direct Assignments" (individual user-to-resource permissions). Identify accounts with "Owner" rights that exceed their daily duties.

Ensure all identities are functional before applying the group logic.

Create a security group named **Senior Admins** and add Joseph Myers as a member using the Portal interface.

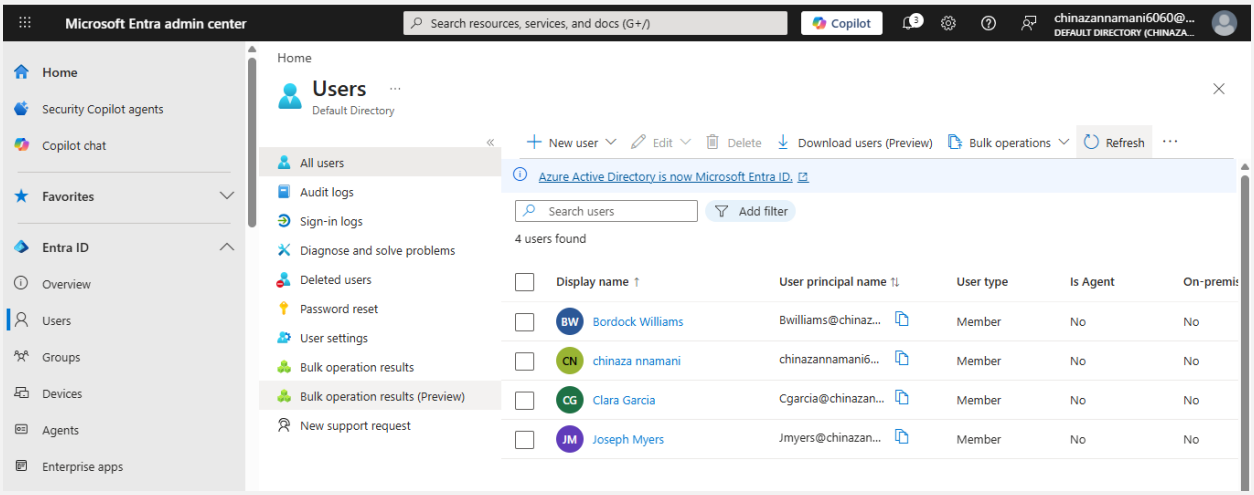
This phase validates the foundational identity management process using the native web interface.

- On the Azure Portal, create a Resource Group named **CloudScale-Logistics-RG** strictly within the **East US region**. This prevents "regional compliance" violations. This also ensures that all resources associated with CloudScale Logistics are created and stored within this region.

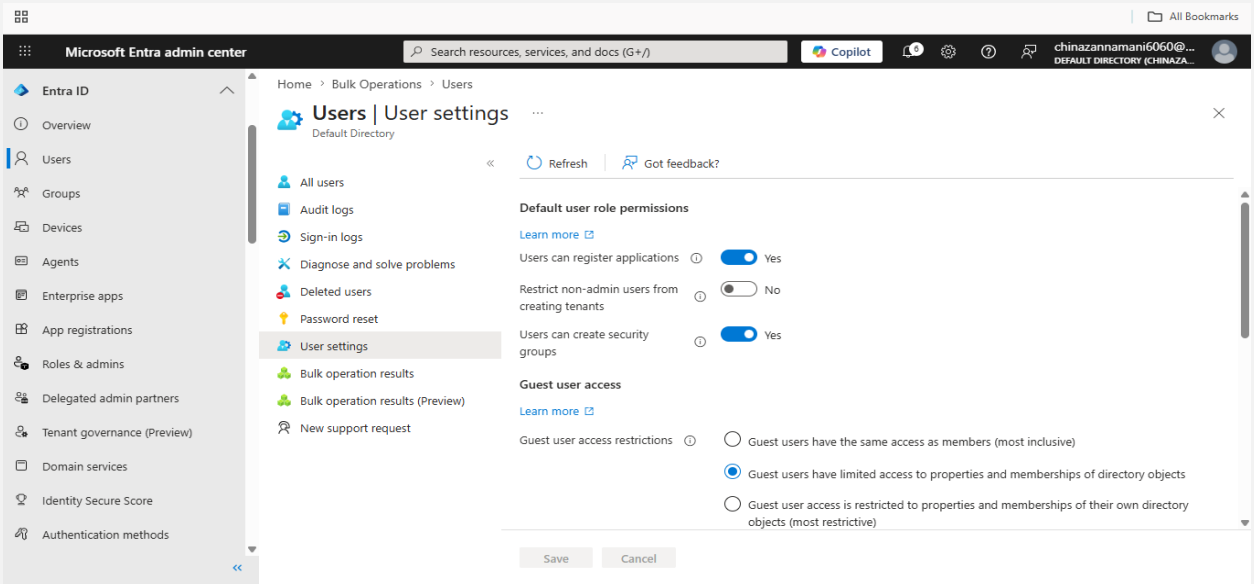


- Navigate to Microsoft Entra ID via the Azure Portal.

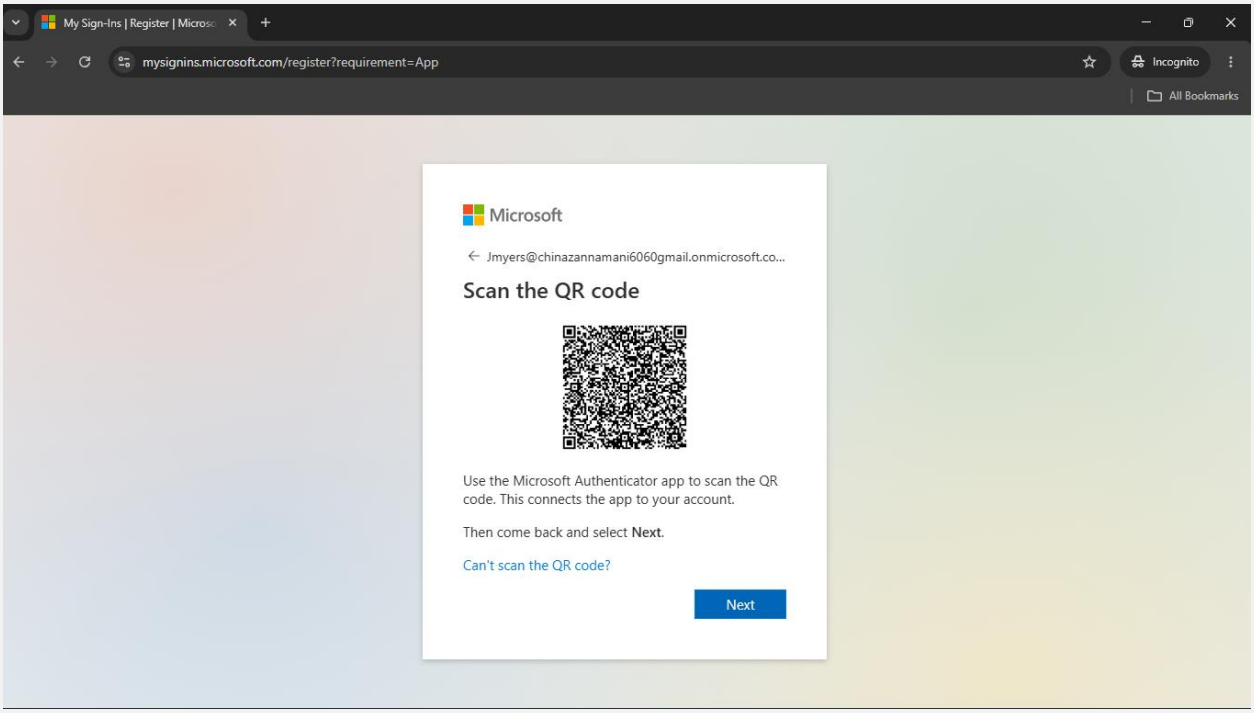
- Provision the user accounts for **Joseph Myers, Clara Garcia, and Bordock Williams**.

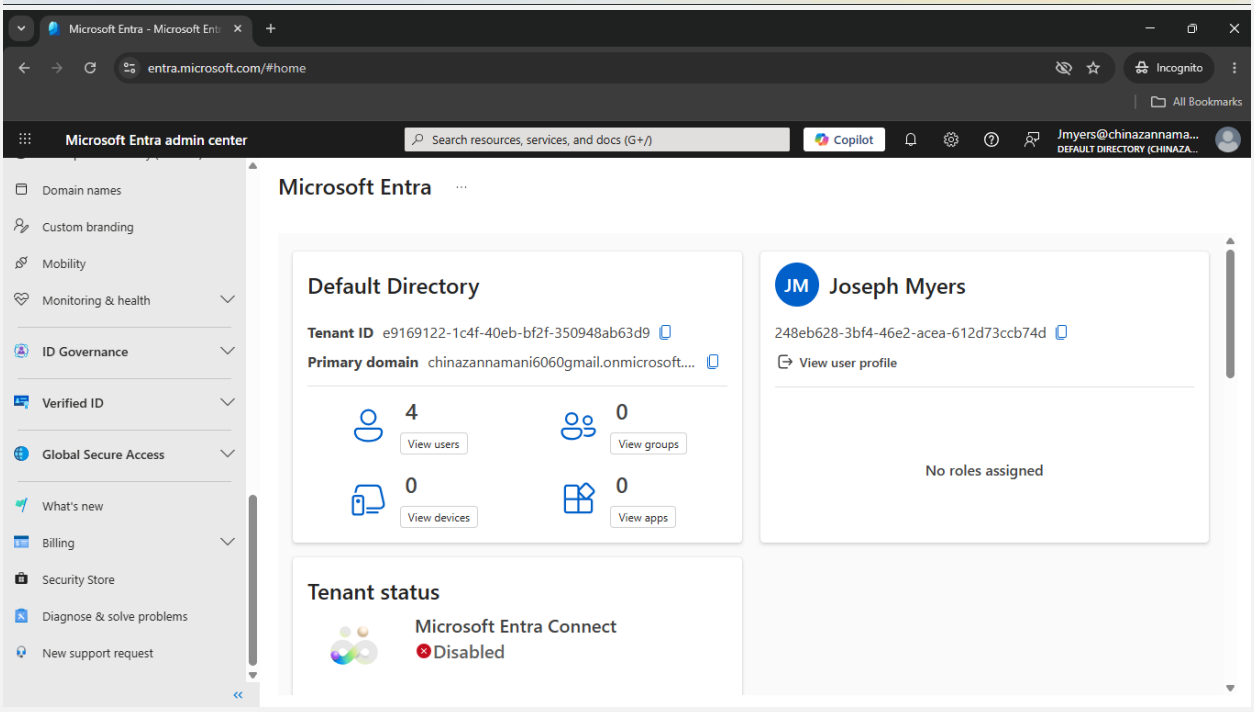


- Permission creep audit: New users in Microsoft Entra ID do not have any administrative roles (like Global Administrator or User Administrator) attached to them by default. However, by default, Entra ID provisions new internal accounts as "Member" users. Member users immediately inherit a baseline set of read and write privileges across the directory. They can: Read the Directory Register Applications Invite External Guests Manage Their Own Profile.

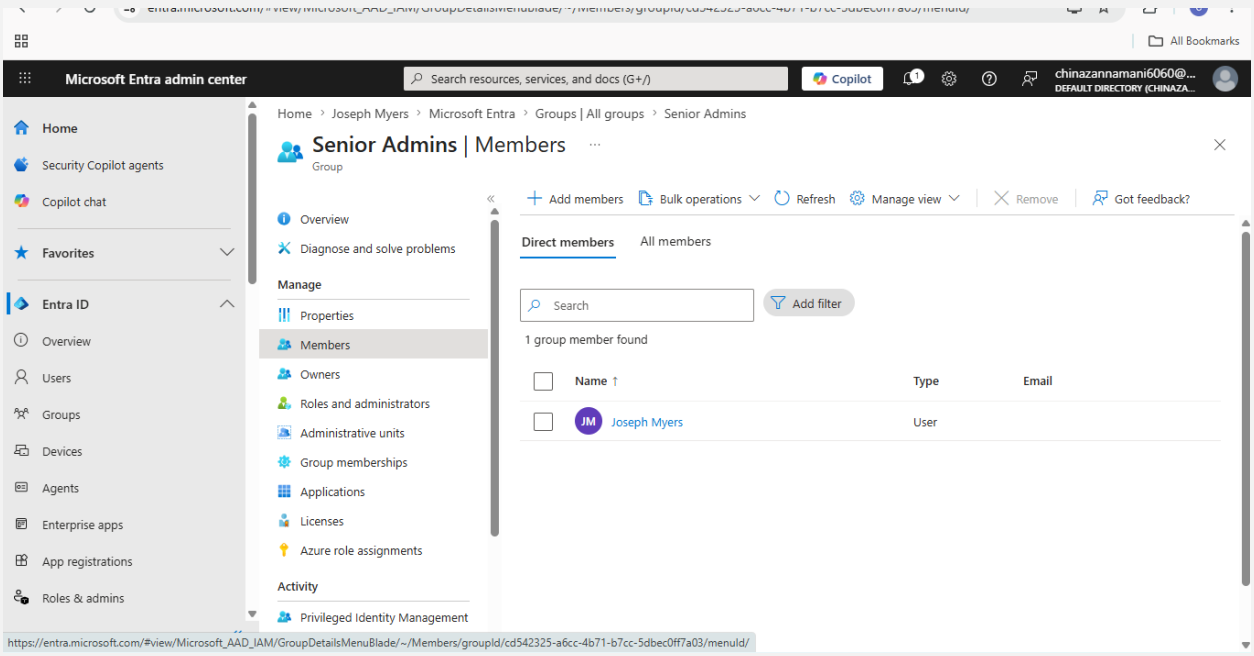


- Verify that all three users can authenticate but currently possess zero access to resources, establishing the "need-to-know" foundational security model while ensuring MFA is enabled.





- Create the security group labeled **Senior Admins**.
- Manually assign the user object to the security group using the portal's graphical assignment flow.



Phase 2 (Scripting): Use PowerShell to programmatically create the Junior Admins group and assign Clara Garcia.

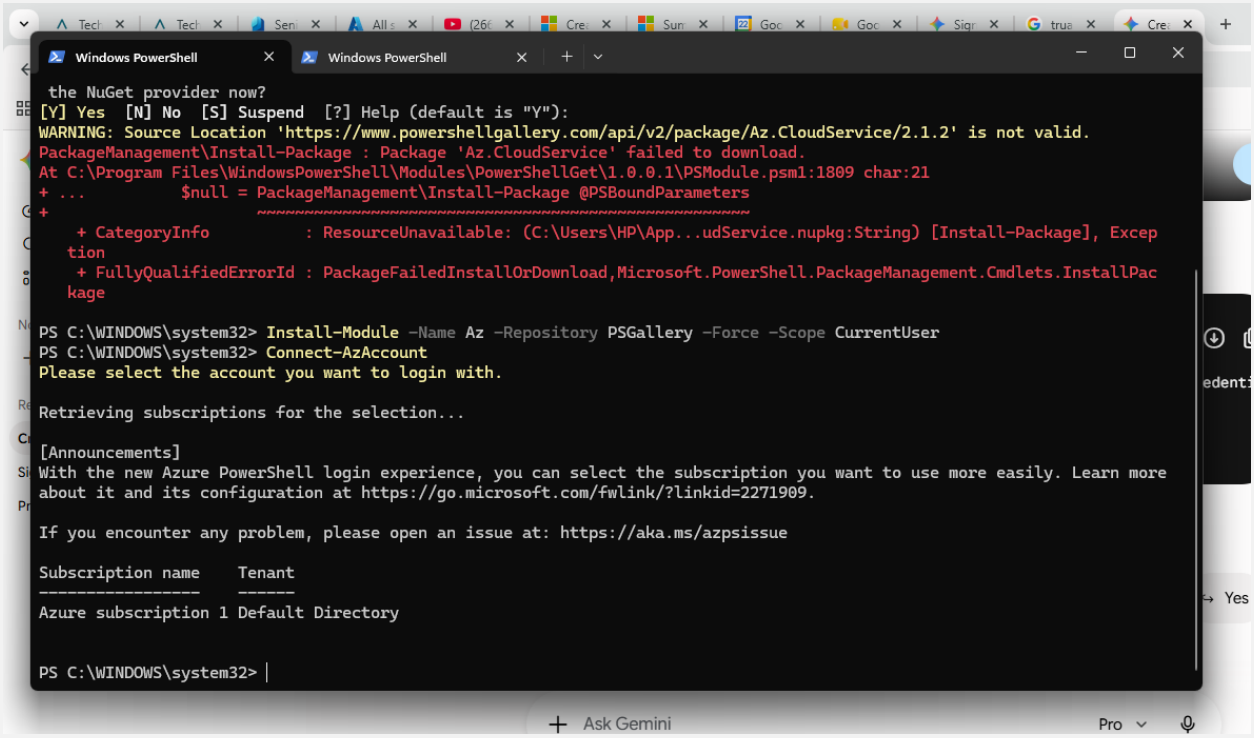
Environment: Azure PowerShell (Az Module)

Objectives:

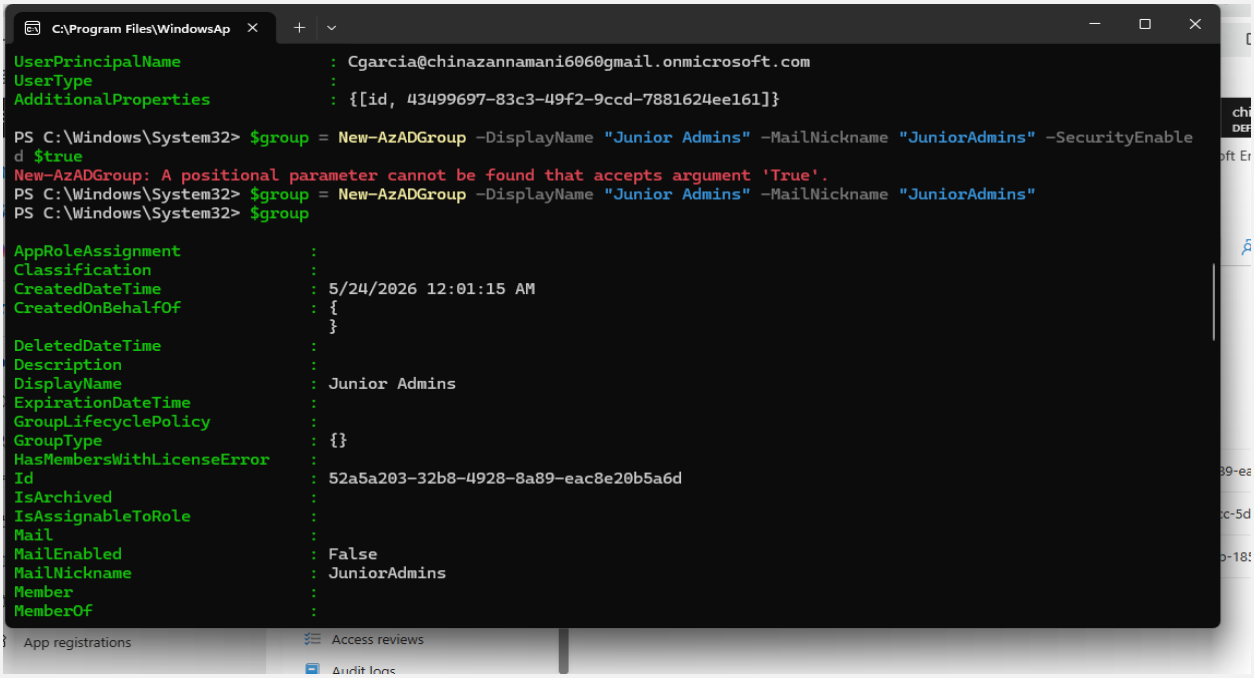
Use PowerShell to create the Junior Admins group and assign Clara Garcia. This validates the ability to scale onboarding via automation.

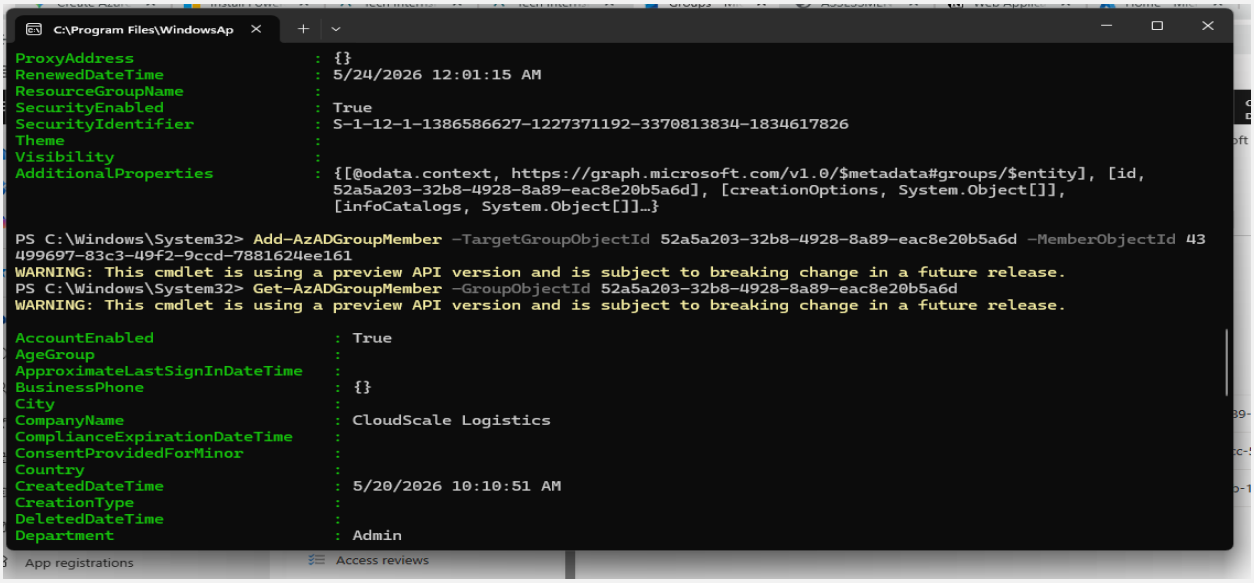
This phase transitions from manual deployment to programmatic execution using native Windows automation.

- Launch a standardized PowerShell environment.



- Instantiate the **Junior Admins** security group.
- Pass the user object ID into the group membership assignment to finalize the programmatic onboarding test.





PowerShell Commands used

Powershell

```
# Install Az module

Install-Module -Name Az -Repository PSGallery -Force -Scope CurrentUser

# Authenticate to the Azure portal

Connect-AzAccount

# Get user object and store in "user" variable

$user = Get-AzADUser -DisplayName "Clara Garcia"

# Create group and store object in "group" variable

$group = New-AzADGroup -DisplayName "Junior Admins" -MailNickname "JuniorAdmins"

# Add user "Garcia" to group

Add-AzADGroupMember -TargetGroupObjectId $group -MemberObjectId $user

# Verify that "Garcia" has been added to "Junior Admins" group

Get-AzADGroupMember -GroupObjectId $group
```

AMDARI

Phase 3 (CLI): Utilize Azure CLI to create the Service Desk group and assign Bordock Williams.

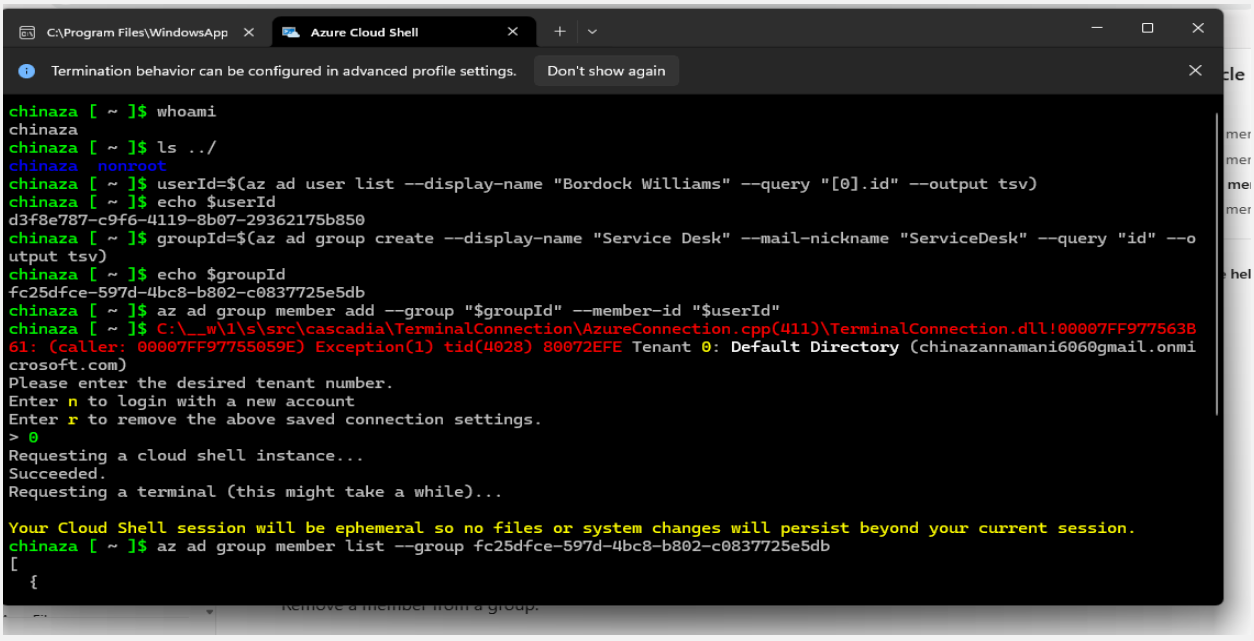
Environment: Azure CLI

Objectives:

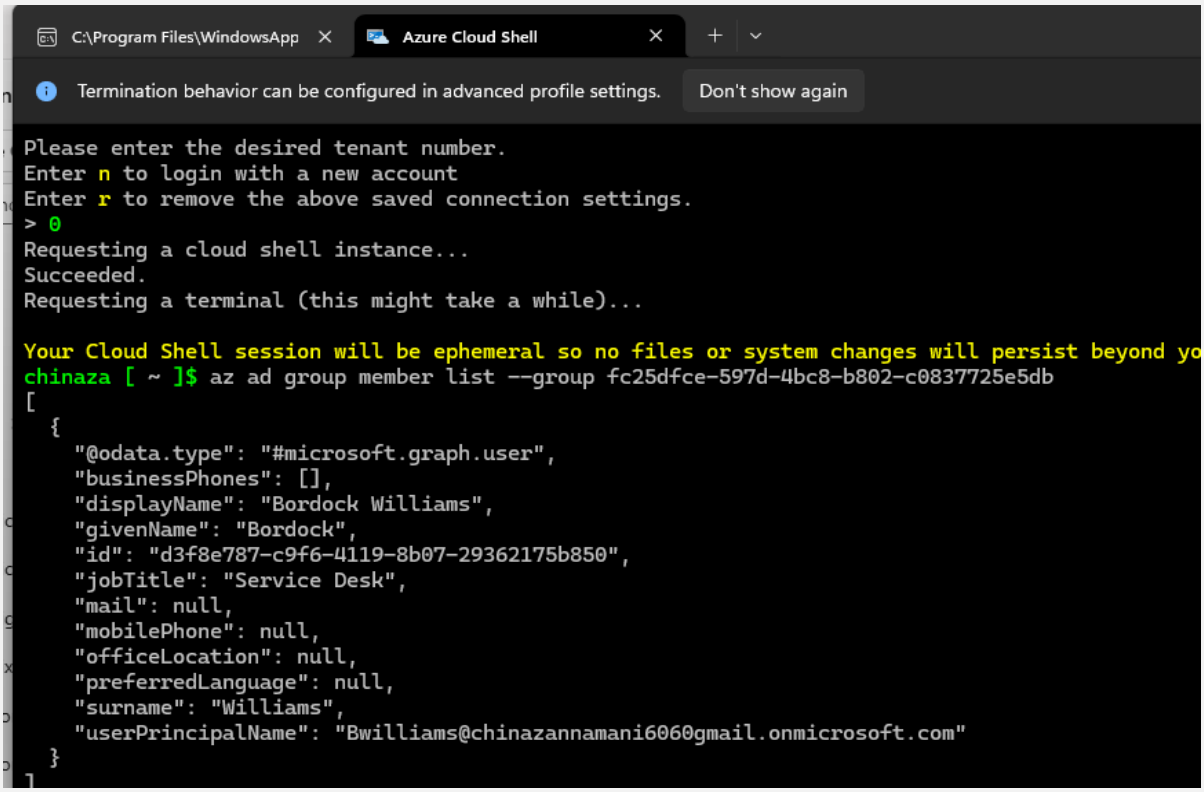
Execute CLI commands to create the Service Desk group and add Bordock Williams. This completes the creation of the three distinct security groups.

This phase validates compatibility with Linux-based management environments, ensuring cross-platform management capabilities.

- Retrieve the existing Object ID for **Bordock Williams** using.
- Create the security group using the Azure CLI without appending invalid OData filtering parameters.
- Assign the member to the group using the retrieved Object IDs.
- Verify successful group population by querying the group membership list.



AMDARI



Azure CLI commands used

Bash

```
# 1. Retrieve Bordock's Object ID

userId=$(az ad user list --display-name "Bordock Williams" --query "[0].id" --output tsv)

# 2. Create the "Service Desk" Security Group and grab its ID

groupId=$(az ad group create --display-name "Service Desk" --mail-nickname
"ServiceDesk" --security-enabled true --mail-enabled false --query "id" --output tsv)

# 3. Add Bordock to the newly created group

az ad group member add --group "$groupId" --member-id "$userId"

# 4. Verify that Bordock is a member of the group

az ad group member list --group "$groupId"
```

Phase 4 (Governance): Assign the Virtual Machine Contributor role to the Service Desk group.

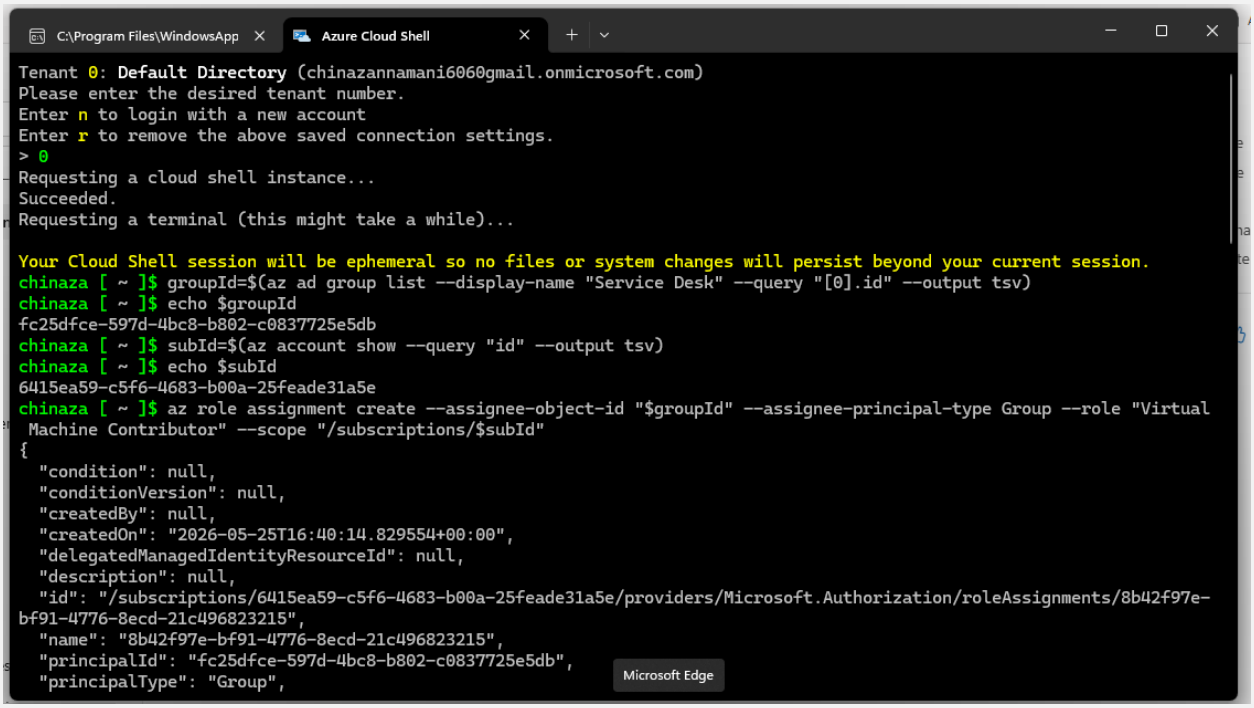
Environment: Azure CLI / Azure Portal

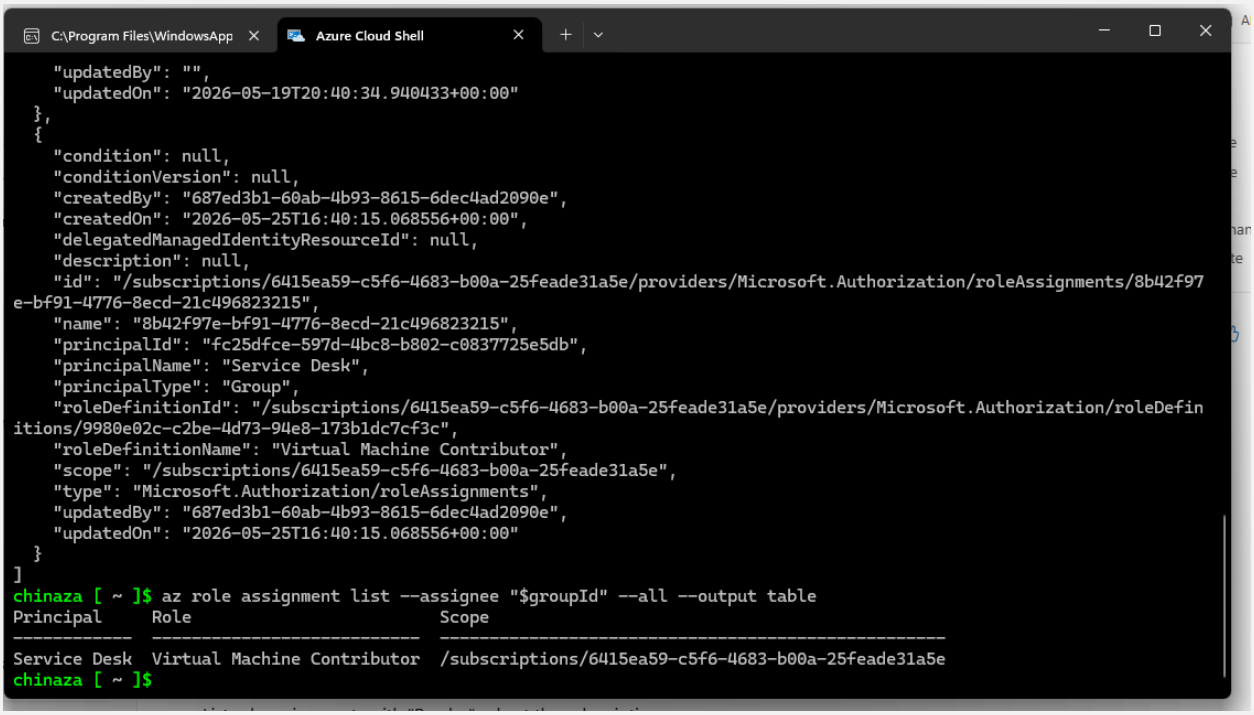
Objectives:

Assign the Virtual Machine Contributor role to the Service Desk group at the resource group level.

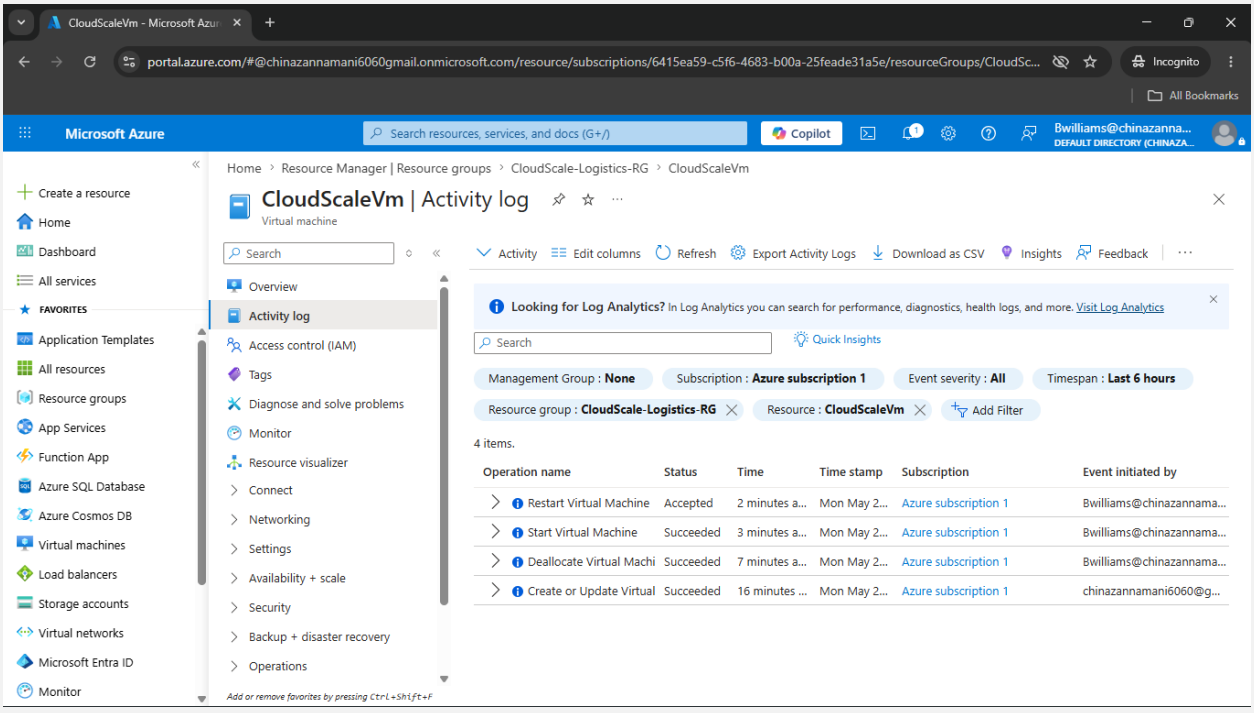
This phase maps directory identities to tangible infrastructure permissions using Azure's RBAC model.

- Identify the target boundary (Subscription or specific Infrastructure Resource Group).
- Assign the **Virtual Machine Contributor** role to the **Service Desk** group Object ID.

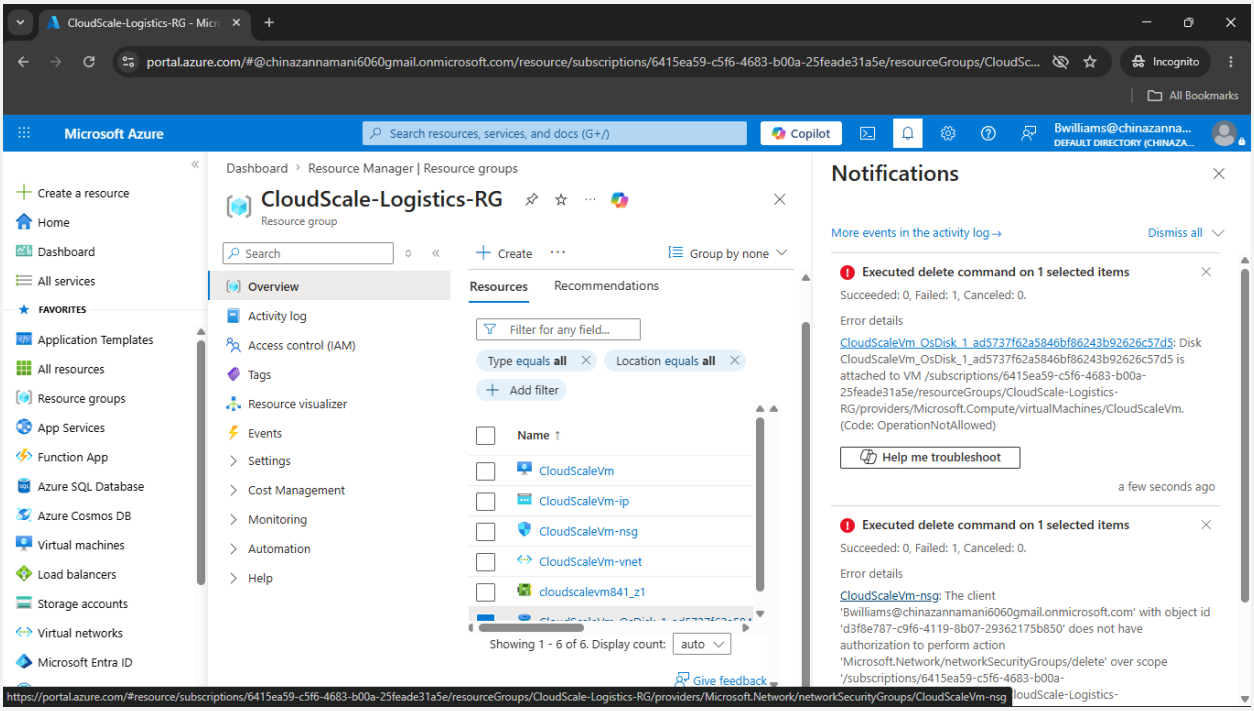




- Validate that members possess the authority to manage VM lifecycles (Start, Stop, Deallocate, Restart).



- Confirm that members are strictly restricted from altering the surrounding virtual networks, modifying storage accounts, or deleting the parent infrastructure.



Azure CLI commands used to assign role

Bash

```
# 1. Fetch the Service Desk group's Object ID

groupId=$(az ad group list --display-name "Service Desk" --query "[0].id" --output tsv)

# 2. Fetch the current active Subscription ID

subId=$(az account show --query "id" --output tsv)

# 3. Create the Role Assignment at the Subscription scope

az role assignment create --assignee-object-id "$groupId" --assignee-principal-type Group --role "Virtual Machine Contributor" --scope "/subscriptions/$subId"

# 4. Verify that the assignment was successfully applied

az role assignment list --assignee "$groupId" --all --output table
```

Phase 5: Resource Decommissioning & Cost Management

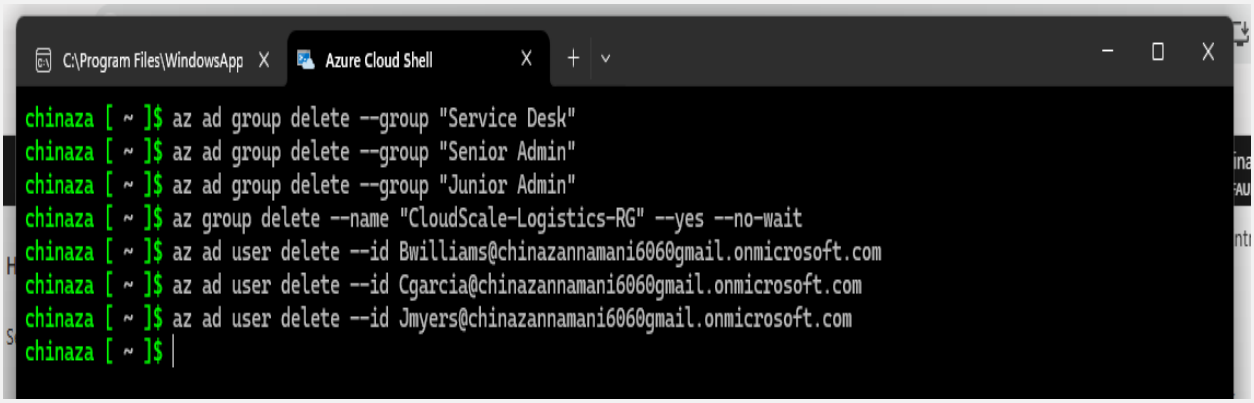
Environment: Azure CLI

Objectives:

Use the Azure CLI to delete the CloudScale-Logistics-RG and all associated test resources. Remove the test users (Joseph, Clara, Bordock) and security groups to ensure no recurring costs or orphaned identities remain in the tenant.

This operational phase ensures the secure and complete removal of all test assets.

- Execute the deletion of the **Infrastructure Resource Group** to immediately halt recurring compute and storage costs.
- Utilize the `--yes` and `--no-wait` parameters to bypass terminal locking during the infrastructure destruction process.
- Purge the Entra ID security groups (Service Desk, Junior Admins, and Senior Admins) to prevent orphaned directory objects.
- Delete the individual test user identities (Joseph, Clara, and Bordock).



Azure CLI commands used

```bash
# Delete groups
az ad group delete --group "Service Desk"
az ad group delete --group "Senior Admin"
az ad group delete --group "Junior Admin"
# Delete the Resource group
az group delete --name "CloudScale-Logistics-RG" --yes --nowait
# Delete users
az ad user delete --id "Bordock's Id"
az ad user delete --id "Garcia's Id"
az ad user delete --id "Joseph's Id"
```

Reflection & Conclusion

This Proof of Concept successfully demonstrated that Azure Identity and Access Management is highly adaptable across varied environments. The Azure Portal provided a baseline for initial configurations, while PowerShell introduced vital automation for scalable user onboarding. Finally, utilizing the Azure CLI in Bash validated cross-platform flexibility, proving that Linux environments can natively manage Entra ID objects.

Beyond identity creation, implementing Role-Based Access Control (RBAC) was a critical success. By assigning the Virtual Machine Contributor role to the Service Desk group, this workflow enforced the principle of least privilege. This allowed essential operational control over VM lifecycles while strictly denying access to underlying storage and virtual networks, minimizing potential security risks.